Generation of Discrete Timed Probabilistic Transition System for Reconcilitation of Simulation and Execution under Timing Uncertainties

Irman Faqrizal, Julien Deantoni

TECHNICAL REPORT
N° ????
February 2026
Project-Teams Kairos

ISSN 0249-0803 ISRN INRIA/RT--????--FR+ENG

Generation of Discrete Timed Probabilistic Transition System for Reconcilitation of Simulation and Execution under Timing Uncertainties

Irman Faqrizal*, Julien Deantoni*

Project-Teams Kairos

Technical Report n° ???? — February 2026 — 21 pages

Abstract: Managing timing uncertainties is critical in safety-critical systems like Software-Defined Vehicles. Simulation-based verification, which is the state-of-the-art timing analysis method, often fails to capture the full range of behaviours observed during deployment. We propose a formal method to generate a Timed Probabilistic Transition System (TPTS) from a network of Discrete Stochastic Timed Automata. By analytically computing transition probabilities over the system's full state-space, our approach provides an exact probabilistic semantics as a "golden" reference model. We demonstrate the effectiveness of this method on an automotive case study, enabling precise reasoning about response times and rare events.

Key-words: Timed automata, Timing uncertainties, Stochastic real-time systems.

* Université Côte d'Azur, Inria, CNRS, i3S, Sophia-Antipolis, France

RESEARCH CENTRE
Centre Inria d'Université Côte d'Azur
2004 route des Lucioles - BP 93
06902 Sophia Antipolis Cedex

Génération d'un système de transition probabiliste à temps discret pour la réconciliation de la simulation et de l'exécution en présence d'incertitudes temporelles

Résumé : La gestion des incertitudes temporelles est cruciale dans les systèmes critiques pour la sécurité, tels que les véhicules à pilotage logiciel. La vérification par simulation, méthode d'analyse temporelle de pointe, peine souvent à appréhender l'ensemble des comportements observés lors du déploiement. Nous proposons une méthode formelle pour générer un système de transition probabiliste temporisée (TPTS) à partir d'un réseau d'automates temporisés stochastiques discrets. En calculant analytiquement les probabilités de transition sur l'ensemble de l'espace d'états du système, notre approche fournit une sémantique probabiliste exacte servant de modèle de référence. Nous démontrons l'efficacité de cette méthode sur une étude de cas automobile, permettant un raisonnement précis sur les temps de réponse et les événements rares.

Mots-clés : Automates temporisés, incertitudes temporelles, systèmes stochastiques temps réel.

Contents

1 Introduction

System design paradigms that rely on hardware abstraction layers, such as Software-Defined Vehicles (SDVs) [18], enable the development of high-level applications without requiring extensive knowledge of the underlying hardware. In practice, developers can specify application requests for data and computation resources. However, this abstraction results in systems with significant timing uncertainties: requested data traverses complex software layers and rarely arrives at a precise time. Moreover, resource competition between concurrent applications further influences execution times. Design-time analysis is therefore vital to ensure that timing properties, such as end-to-end response times, are strictly satisfied.

Simulation-based techniques are the current state-of-the-art for analysing the impact of these uncertainties. For a vehicle application, developers typically specify best- and worst-case execution times for software components, alongside the timing behaviour of sensor data and actuator access, often represented as periodicities with jitters. Both execution times and periodicities are described as time bounds with probabilistic distributions. The resulting specification is then simulated to estimate timing properties of interest.

The primary limitation of simulation-based analysis is that actual system execution upon deployment may differ significantly from simulation results. In such cases, it is difficult to determine whether the simulation was poorly calibrated or if the implementation deviates from the specification. Addressing this requires a formal model to serve as a *golden reference* for comparing simulation and execution results. Such a reference must be formally generated from the specification to ensure its reliability as a ground truth for analysing the impact of timing uncertainties.

In this paper, we propose a method to generate a formal model from a specification of a system subject to timing uncertainties. Specifically, the specification is described as a network of **Discrete Stochastic Timed Automata (DSTA)**, and the generated model is a **Timed Probabilistic Transition System (TPTS)**. Our approach leverages the `IF` toolset [2] to first compute the discrete state-space of the networked automata as a Timed Labelled Transition System (TLTS).

We then define a semantic mapping $\mathcal{M}$ that transforms the global TLTS into a TPTS. This is achieved by resolving the non-deterministic interleavings in the underlying TLTS through a probability measure $\mathcal{P}$, which is analytically derived from the stochastic delay distributions specified in the DSTA. The transition probabilities in the resulting TPTS are computed by solving a system of linear constraints that represent the synchronization and interleaving of local behaviours, ensuring global consistency with local stochastic delays.

The main contribution of this work is the automated generation of TPTSs from networked DSTA. We demonstrate that these TPTSs can be quantitatively compared with models produced by injecting simulation or execution traces into the same TLTS structure. This method identifies the probabilities of edge cases not fully covered by finite trace sets. For instance, if a specific response time has zero probability during simulation, the reference TPTS can be queried to determine its theoretical expected probability. Such analysis is performed using action-based probabilistic model checking [14] via the `CADP` toolbox [9].

Systems with timing uncertainties are formally categorized as stochastic real-time systems. While several verification frameworks exist, the most relevant is the `mcsta` tool [10] within the Modest toolset [11], which operates on Stochastic Timed Automata (STA) [6]. Our DSTA model is a timing-centric subset of STA designed for exact analytical tractability. We adopt discrete-time semantics and isolate stochastic behaviour by assuming that each location possesses a unique outgoing edge, focusing the analysis on the probability distributions of the delays rather than probabilistic branching.

While `mcsta` generates explicit state-based representations using Markov Decision Processes (MDPs), our TPTS-based approach is specifically designed to facilitate the comparison of models derived from empirical traces. Unlike MDPs, which model timing uncertainties as probabilistic choices on random variables, TPTSs map directly to observable action sequences. This alignment makes the injection of timestamped execution traces–which do not inherently contain random variables–significantly more straightforward in our framework.

The paper is organized as follows. Section 2 introduces preliminary notions. Section 3 surveys related works. Section 4 explains the generation of TPTSs from DSTA networks. Section 5 discusses implementation, followed by a vehicle application case study in Section 6. Section 7 concludes the paper.

2 Preliminaries

The following notations are used throughout the remainder of this paper:

- Let X be a finite set of discrete clocks. Each clock $c \in X$ takes its value in $\mathbb{N}$, and increases by one unit on each clock tick, unless it is reset.
- b_X is a predicate over a clock $c \in X$. Predicates are of the form $x \bowtie c$, where $x \in X$, $c \in \mathbb{N}$, and $\bowtie \in \{<, \leq, =\}$.
- For a clock $x \in X$, the notation y_x denotes the reset operation $x := 0$. Multiple clock resets may be applied simultaneously.
- Let A be a set of actions and C a set of clock tick labels. The set of labels is defined as the disjoint union $L = A \uplus C$. A function $V : L \to \mathbb{N}$ assigns to each label the number of clock ticks elapsed by the system when the corresponding transition is taken. For all $l \in A$, $V(l) = 0$; labels $l \in C$ satisfy $V(l) > 0$.
- $\mathbb{F}$ is a set of fractional number sequences used to annotate the predicate b_x in a timed automaton. Every $F \in \mathbb{F}$ is a sequence $f^0, f^1, ..., f^n$ with $f^i \in \mathbb{Q}_{>0}$ for all i. The length of the sequence is determined by the clock constraint. If b_X restricts clock c to the domain $[0; n[$, then the sequence F contains n values, exactly one probability value for each admissible valuation of c. Additionally, each sequence F is such that $\sum_{i=0}^{n} f_i = 1$.
- Let $\mathbb{E}$ be a set of equations over the variables and functions introduced above.

Timed Automata. Timed Automata (TA) [1] is an automata-based specification language for real-time systems.

Definition 2.1 (Timed Automata)**.** A timed automaton is a tuple $(O, o^0, A, X, B_X, Y_X, E)$ where:

- O is a finite set of locations, with initial location $o^0 \in O$,
- A is a finite set of actions,
- X is a finite set of discrete clocks,
- B_X is a set of conjunctive clock constraints over X,
- Y_X is a set of clock reset operations, and
- $E \subseteq O \times A \times B_X \times Y_X \times O$ is a set of edges.

In this paper, we consider timed automata with discrete clocks that are *deterministic*: for every location and every clock valuation, at most one outgoing edge is enabled. All transitions are assumed to be time-delayable as long as their guard remains satisfied. Under these assumptions, invariants on locations are not required.

Timed Labelled Transition System. A Timed Labelled Transition System (TLTS) is a transition system in which each transition is labelled by either an action or a clock tick. We utilise the `IF` toolset [2] to generate TLTSs from TA and networks of TA.

Definition 2.2 (Timed Labelled Transition System)**.** A timed labelled transition system is a tuple (S, s^0, L, T), where:

- S is a set of states, with an initial state $s^0 \in S$,
- L is a set of labels (containing both action and time-related labels), and
- $T \subseteq S \times L \times S$ is a set of transitions.

Let $h = (S, s^0, L, T)$ be a TLTS. A (finite or infinite) path of h is a sequence $q = s_0 \xrightarrow{l_1} s_1 \xrightarrow{l_2} s_2 \xrightarrow{l_3} \cdots$ such that $s_0 = s^0$ and $(s_{k-1}, l_k, s_k) \in T$ for all k. The set of all paths of h is denoted by Q_h.

A transition $s \xrightarrow{c_a} s' \in T$ is called a delay transition for action $a \in L$ if $\exists s \xrightarrow{a} s'' \in T \wedge V(c_a) = 1$. We write $delays : Q \to \mathbb{N}$ to get the number of delay transitions in a given path. A transition $s \xrightarrow{a^i} s' \in T$ is called an action transition at time i if $\exists s_1 \xrightarrow{c_a^1} s_2, s_2 \xrightarrow{c_a^2} s_3, ..., s_i \xrightarrow{c_a^i} s \in T$.

Discrete Stochastic Timed Automata. In our approach, systems are specified using a network of Discrete Stochastic Timed Automata (DSTA).

Definition 2.3 (Discrete Stochastic Timed Automata)**.** A discrete stochastic timed automaton $(O, o^0, A, X, B_X, Y_X, D, E)$ is a timed automaton extended with a probabilistic distribution labelling $D : E \to \mathbb{F}$.

Timed Probabilistic Transition System. A Timed Probabilistic Transition System (TPTS) is a TLTS extended with probabilities on its transitions.

Definition 2.4 (Timed Probabilistic Transition System)**.** A TPTS (S, s^0, L, P, T) is a TLTS extended with a probability labelling $P : T \to (0, 1]$, such that $P(s \xrightarrow{l} s')$ is the probability of transition $s \xrightarrow{l} s' \in T$ to be traversed and, $\forall s \in S$, $\sum_{s \xrightarrow{l} s'} P(s \xrightarrow{l} s') = 1$.

3 Related Work

The notion of Stochastic Timed Automata (STA) is first introduced in [6] as the operational semantics of the `Modest` language. Essentially, STA is TA extended with random variables and edge probabilities. The random variables can be specified using probabilistic distributions to constrain clocks on the transitions. This enables the specification of transition delay distributions. STA also permits assigning probabilities to the edges as in Probabilistic Timed Automata (PTA). Our DSTA model can be seen as a subset of STA. For specifying the transition delay distribution, we choose to characterise the clock constraints with a probabilistic distribution directly without random variables. Moreover, we consider discrete clocks to allow specifying the delay distribution as a finite sequence of fractional numbers. Currently, our state-space generation approach does not support probabilities on the edges.

In [10], the authors propose `mcsta`, an explicit-state model checker tool for STA, as part of the `Modest` toolset [11]. The main idea implemented in the tool is to translate STA into PTA by assigning probabilities to the transitions according to the distribution of the random variables. The digital clock approach is then used to convert PTA into a Markov Decision Process (MDP),

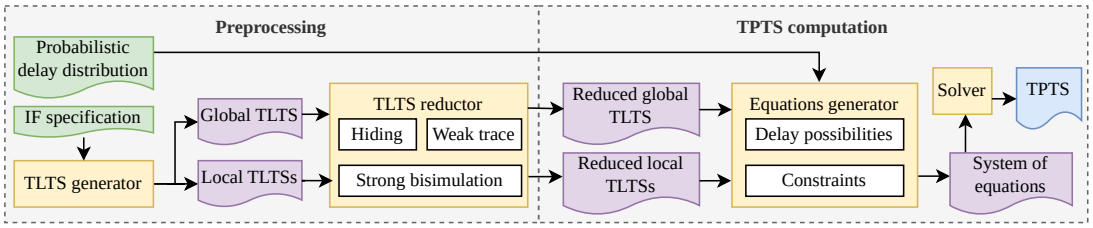


Fig. 1: Overview of the approach

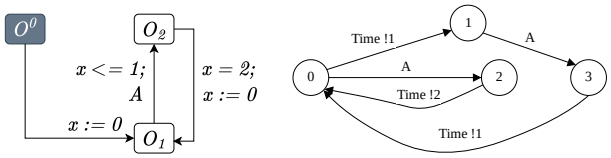
which can be model checked using value iteration techniques [5]. The work in [12] extends the tool with a method to store the state space on the hard drive during verification, enabling the handling of models with a large number of states. Their representation of a state-space with MDP involves two types of transition: (i) transitions with probabilities dedicated to choosing the values of random variables and (ii) non-deterministic transitions for choosing actions that can happen at a given time. In comparison, there is only one type of transition in our discrete TPTS model, representing a possibility to either trigger an action or not. In the former case, the transition is labelled by the action, whereas in the latter case, the transition is labelled by a clock tick indicating time is passing. If more than one action is available at a given time, then probabilities are distributed fairly according to the specification.

The `UPPAAL-SMC` tool introduced in [4] provides an alternative method for model checking STA by simulating the specification rather than numerical analysis. This method is known as statistical model checking, and the work in [20] is one of the earliest notable works that proposed it. In `UPPAAL-SMC`, a system is specified as a `UPPAAL` stochastic model, which is essentially a network of TA enriched with probabilistic distributions of the transition delays. The model checker verifies a given property by simulating the specification, and statistically approximates the probability that the property is satisfied. In our work, we prefer to generate the state-space of the model that can be checked on given properties without resorting to simulation. This is chosen because simulation-based techniques are already commonly used for analysing the type of system that we are targeting, such as software-defined vehicles. Our idea is to generate the state-space that completely represents the specification to serve as a reference model when analysing the simulation results.

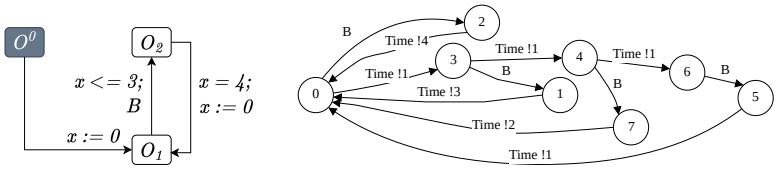
4 State-Space Construction of DSTA

The construction of a TPTS from a DSTA specification follows the workflow illustrated in Fig. 1. In the preprocessing phase, we utilize the `IF` toolset to derive the global TLTS of the network alongside the local TLTS for each constituent automaton. Subsequently, the `CADP` toolbox is employed to refine these transition systems. First, internal operations (e.g., clock initializations) are abstracted by hiding their labels under the silent action τ. We then apply weak trace equivalence reduction to eliminate these τ-transitions and determinize the TLTS. Finally, we apply a reduction based on strong bisimulation that preserves the fundamental properties of time determinism and time additivity [17]. This step ensures a minimal state-space representation while maintaining the precise timing semantics required for subsequent probabilistic analysis.

The TPTS computation phase centers on the derivation and resolution of a linear system of equations. This process takes the reduced TLTSs and the DSTA's stochastic delay distributions as primary inputs. The system is constructed from two distinct sets of constraints. The first set is derived from observational equivalence (detailed in Section 4.1), which aligns the global



(a) Timed automaton A and its (local) TLTS



(b) Timed automaton B and its (local) TLTS

Fig. 2: Example of TA and their corresponding local TLTSs

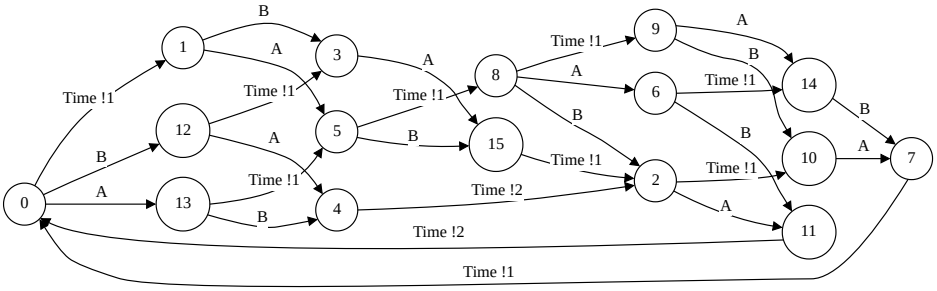


Fig. 3: Example of global TLTS composed from TA A and B

behaviours with the local specifications. The second set enforces probabilistic consistency and timing semantics: it ensures that transition probabilities sum to 1, applies the product rule for synchronous time progression, and preserves action ratios inherited from the underlying timed races, resolved using the probability distributions of the discrete delays. In this system, each variable represents an unknown transition probability in the global TLTS. By solving this system of constraints, we yield the exact values required to transform the TLTS into a valid TPTS.

Example. A network of two DSTA is used to illustrate our approach in the rest of this section. The first automaton in Fig 2a described a periodic action A that is triggered every 2 ticks with a possibility to be delayed by 1 tick. The second automaton in Fig 2b describes a similar behaviour, and the only differences are that the periodicity is 4 ticks and the possibility of being delayed by 1 to 3 ticks. In the generated TLTSs, discrete time progress is represented by the label *Time* $!n$ ($n \in \mathbb{N}$), following the CADP data-parameterized convention. This encoding compresses the state-space and enables direct arithmetic operations on time within verification queries The generated TLTSs of these automata show exactly the described behaviours. For instance, in the TLTS of the first automaton, from the initial state, it is possible to either trigger A or to delay A by 1 tick before finally triggering it. The global TLTS, which is the product of the two automata synchronized on time, is shown in Fig. 3.

4.1 Observational Equivalence

The key part of our approach is observing the probability equivalence between the local TLTSs generated from individual automata and the global TLTS obtained from the network of automata composition.

In order to compute the probabilities of the global TLTS that preserve such equivalence, let π_i be a projection operator that maps a global execution path $\hat{q} \in Q_{\text{global}}$ to its local restriction on a local TLTS A_i with a set of labels L_i. Formally, for any global path $\hat{q} = (l_1, l_2, \ldots, l_n)$, $\pi_i(\hat{q})$ is defined as the subsequence obtained by filtering out all labels $l_j \notin L_i$.

For a local path $q \in Q_{A_i}$, we define its inverse projection relative to the global TLTS H_{global} as the set of all global interleavings that are observationally equivalent to q from the perspective of A_i:

$$\pi_i^{-1}(q) = \{\hat{q} \in Q_{H_{\text{global}}} \mid \pi_i(\hat{q}) = q\} \tag{1}$$

Each $\hat{q} \in \pi_i^{-1}(q)$ represents a distinct global scenario where the sequence of local actions and time advancements matches q, while accounting for the occurrence and interleaving of actions from other automata in the network.

The core of our methodology lies in establishing a formal stochastic equivalence between the local TLTSs and the composed global TLTS. We leverage this relationship to derive a system of linear equations based on the principle of probability conservation. Specifically, the marginal probability of a local path q must be partitioned across all its corresponding global interleavings:

$$P(q) = \sum_{\hat{q} \in \pi_i^{-1}(q)} P(\hat{q}) \tag{2}$$

By enforcing this equivalence for every path q in each local TLTS, we ensure that the resulting TPTS is stochastically consistent with the original DSTA specification.

Technically speaking, the equivalence is implemented by extracting local paths that characterize the occurrence of each action. In each local TLTS, we first identify the *starting state* of an action, which corresponds to the earliest time at which that action can be triggered.

Definition 4.1 (Action starting state)**.** A state s_a is the starting state of action a iff $\exists s_a \xrightarrow{a} s \in T$ and $\nexists s' \xrightarrow{c_a} s_a \in T$.

Assuming actions are unique within each automaton, there exists a unique s_a for every $a \in L_i$. We then consider the set of paths originating from s_a and terminating with the execution of a. We define the function $\mathit{localPaths} : A \times H \to 2^Q$ to perform this extraction. For a given action a in a local TLTS h, it returns a set of paths $Q_a = \{q_{a,0}, q_{a,1}, \ldots, q_{a,n}\}$, where each $q_{a,i}$ is a sequence of the form $s_a \xrightarrow{\text{Time!1}} s_1 \ldots \xrightarrow{a} s_m$ containing exactly i time increments (i.e., for which $\mathit{delays}(q_{a,i}) = i$).

To enforce stochastic consistency, we map these local observations to the global model. For every local path $q \in \mathit{localPaths}(a, h)$, we identify its counterparts in the global TLTS using the inverse projection $\pi_i^{-1}(q)$ defined previously.

Formally, the function $\mathit{globalPaths} : Q \times H \to 2^{S \times 2^Q}$ partitions the inverse projection of a local path based on the global starting states. For a local path q, it identifies the set of pairs (s, Q^s) such that:

$$\mathit{globalPaths}(q, H_{\text{global}}) = \{(s, \hat{Q}_s) \mid s \in S_{\text{global}}, \hat{Q}_s = \hat{q} \in \pi_i^{-1}(q) \mid \text{start}(\hat{q}) = s\} \tag{3}$$

where $\hat{Q}_s \neq \emptyset$. This ensures that the probability conservation $P(q) = \sum P(\hat{q})$ is applied correctly at each possible entry point in the global system, accounting for the fact that the same local behaviour q might be reachable from multiple global states s.

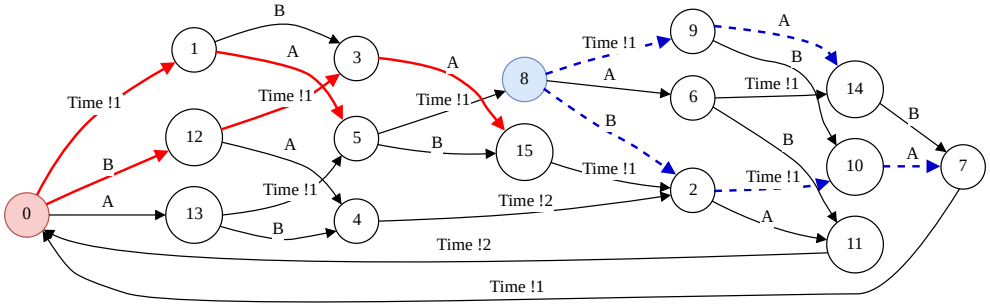


Fig. 4: The sets of paths equivalent to the path $0 \xrightarrow{Time\ !1} 1, 1 \xrightarrow{A} 3$

In practice, this computation retrieves all global paths where the sequence of labels, when filtered by π_i, matches the sequence of q. This includes all interleavings where any number of foreign actions $a' \notin L_i$ occur between the local steps of q, provided they do not alter the local state progression of A_i. The resulting sets of global paths are then used to populate the system of linear equations.

Example. Suppose we want to find the equivalences of all paths in the automaton with the action A (Fig. 2a). Then, the action starting state is the initial state 0 because it has an outgoing transition labelled by A and no incoming delay transition. The set of local paths $\{(0 \xrightarrow{A} 2), (0 \xrightarrow{Time\ !1} 1, 1 \xrightarrow{A} 3)\}$ can thus be extracted. Afterwards, we find the equivalence of every path in the global TLTS. For instance, the path $0 \xrightarrow{Time\ !1} 1, 1 \xrightarrow{A} 3$ is equal to the set of paths highlighted in Fig. 5 grouped by source states.

4.2 System of Equations

The TPTS computation takes as input the probabilistic distributions of the delays. This input in our DSTA model is the labelling function $D : E \to \mathbb{F}$. To simplify the algorithm, we associate the probabilistic distribution described in $F \in \mathbb{F}$ with actions on the edges of the DSTA. Therefore, this input is written as $D = \{(a_1, F_{a_1}), (a_2, F_{a_2}), ..., (a_n, F_{a_n})\}$ instead, where $F_{a_i} = f^0_{a_i}, f^1_{a_i}, ..., f^j_{a_i}$ is the sequence of fractional numbers representing the probabilistic distribution of action a_i to happen within $0, j \in \mathbb{N}$.

Algorithm 1: Generate system of equations

Inputs : h_{gt}, H_{lt}, D
Output: Eqs
1 **foreach** $(a, F) \in D$ **do**
2 $h := getTLTS(a, H_{lt})$
3 $Q_{lt} := localPaths(a, h)$
4 **foreach** $q \in Q_{lt}$ **do**
5 $f := getProb(F, q)$
6 **foreach** $(s, Q_{gt}) \in globalPaths(q, h_{gt})$ **do**
7 $Eqs := Eqs \cup buildEq(Q_{gt}, f)$

Algorithm 1 describes how the system of equations is generated. It takes as input the global

Table 1: Rules for generating constraint equations

(A) $\forall s \in S,\ \sum_{s \xrightarrow{l} s'} P(s \xrightarrow{l} s') = 1$	**(B)** $\forall s \in S, \frac{P(s \xrightarrow{a_1^i} s')}{P(s \xrightarrow{a_2^j} s'')} = \frac{f_{a_1}^i}{f_{a_2}^j}$
(C) $\forall s \in S,\ P(s \xrightarrow{c_A} s') = \prod_{s \xrightarrow{a^i} s'} 1 - \phi_a^i, a \in A$	

TLTS h_{gt}, local TLTSs H_{lt}, and a set of probabilistic delay distributions D. It returns a set of equations *Eqs*.

For brevity, we use the following functions:

- $getTLTS : A \times 2^H \rightarrow H$ to find a TLTS that contains a specific action, i.e., $getTLTS(a, H) = (S, s^0, L, T), a \in L$
- $getProb : 2^F \times Q \rightarrow \mathbb{Q}$ to find a probability according to the number of delay transitions in a path, i.e., $getProb(F, q) = f_i \in F,\ i = delays(q)$
- $buildEq : 2^Q \times \mathbb{Q} \rightarrow \mathbb{E}$ to create an equation that sums up the multiplication product of every path in the set of paths and assigns a fractional number as the result, e.g., for $Q = \{(s_1 \xrightarrow{a} s_2), (s_1 \xrightarrow{c_a^1} s_3, s_3 \xrightarrow{a} s_4)\}$ and $f = \frac{1}{2}$, then $buildEq(Q, f) =$ "$\mathbf{P(s_1 \xrightarrow{a} s_2)\ +\ P(s_1 \xrightarrow{c_a^1} s_3)\ *\ P(s_3 \xrightarrow{a} s_4)\ =\ \frac{1}{2}}$"

The algorithm first iterates through the set of distributions and finds the local TLTS containing each action (lines 1 and 2). Then, the set of paths corresponding to different possibilities for executing the action is extracted from the local TLTS (line 3). For every possible path (line 4), its probability is obtained (line 5), and the set of paths that match its sequence of actions is extracted (line 6). Finally, an equation is created and added to the system for each group of paths with the same starting state (line 7).

In addition to equations based on observational equivalence, we must also include equations to constrain the values of the transition probabilities. For a given global TLTS (S, s^0, L, T) and probabilistic distribution of delays in $D = \{(a_1, F_1), (a_2, F_2), ..., (a_n, F_n)\}$, these additional equations are generated according to the set of rules in Table 1.

Function $\Phi(f^i) = \frac{f^i}{\prod_{j=0}^{i-1} 1 - \Phi(f^j)}$, which maps every $f^i \in F \in \mathbb{F}$ into $\phi^i \in \mathbb{Q}$, is defined to explain rule **(C)**. For an action a and its probabilistic distribution $f_a^0, f_a^1, ..., f_a^n$, the function returns $\phi_a^i \in \phi_a^0, \phi_a^1, ..., \phi_a^n$, which represents the probability of action a to be triggered given that it has been delayed i times.

The rules in Table 1 are described as follows:

(A) Probabilities of transitions outgoing from the same state must sum to 1. This rule is necessary to satisfy the definition of TPTS (i.e., Definition 2.4).

(B) Probabilities of action transitions outgoing from the same state must be according to the ratio of probabilities in the specification. This rule ensures fairness when several transitions labelled with different actions outgoing from the same state (due to interleavings between actions).

(C) Every delay transition probability is the product of probabilities to delay all the actions in the source state. This rule is important because every delay transition in the global TLTS represents a possibility to delay all the available actions on the source state.

Generating equations based on these rules is straightforward. For instance, equations for rule **(A)** are generated by iterating through all states in the TLTS while creating an equation that constrains the probabilities of all outgoing transitions to be equal to 1. Once all the equations are collected, a system of equations solver is used to obtain the values of the variables, thus

Table 2: Fragment of a system of equations

Origin	Equation
Equivalence	$\{\mathbf{P(0 \xrightarrow{A} 13)} + \mathbf{P(0 \xrightarrow{B} 12)} * \mathbf{P(12 \xrightarrow{A} 4)} = \frac{1}{4},\ \mathbf{P(0 \xrightarrow{B} 12)} * \mathbf{P(12 \xrightarrow{Time\ !1} 3)} * \mathbf{P(3 \xrightarrow{A} 15)} + \mathbf{P(0 \xrightarrow{Time\ !1} 1)} * \mathbf{P(1 \xrightarrow{B} 3)} * \mathbf{P(3 \xrightarrow{A} 15)} + \mathbf{P(0 \xrightarrow{Time!1} 1)} * \mathbf{P(1 \xrightarrow{A} 5)} = \frac{3}{4},\ ...\}$
Rule **(A)**	$\{\mathbf{P(0 \xrightarrow{Time\ !1} 1)} + \mathbf{P(0 \xrightarrow{A} 13)} + \mathbf{P(0 \xrightarrow{B} 12)} = \mathbf{1},\ \mathbf{P(1 \xrightarrow{A} 5)} + \mathbf{P(1 \xrightarrow{B} 3)} = \mathbf{1},\ ...\}$
Rule **(B)**	$\{\frac{\mathbf{P(0 \xrightarrow{B} 12)}}{\mathbf{P(0 \xrightarrow{A} 13)}} = \frac{1}{2},\ \frac{\mathbf{P(1 \xrightarrow{B} 3)}}{\mathbf{P(1 \xrightarrow{A} 5)}} = \frac{2}{3},\ ...\}$
Rule **(C)**	$\{\mathbf{P(0 \xrightarrow{Time\ !1} 1)} = \frac{21}{32},\ \mathbf{P(8 \xrightarrow{Time\ !1} 9)} = \frac{1}{2},\ ...\}$

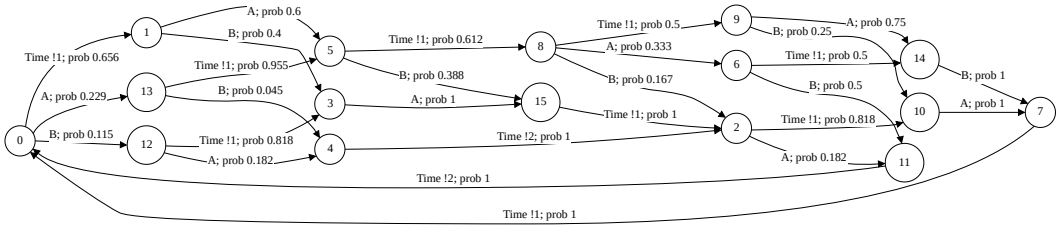


Fig. 5: Example of TPTS

producing the TPTS.

Example. Let us consider the specified probabilistic distributions of the transition delays as $D = \{(A, (\frac{1}{4}, \frac{3}{4})), (B, (\frac{1}{8}, \frac{1}{2}, \frac{1}{8}, \frac{1}{4}))\}$. A fragment of the system of equations collected using our approach is presented in Table 2. The first equivalence equation corresponds to the possibility of action A being triggered without delay; thus, the sum of probabilities associated with the paths in the equation must equal $f_A^0 = \frac{1}{4}$. The first equation in rule **(A)** constrains the sum of transition probabilities from state 0 to equal 1. Equations in rule **(B)** ensure the fairness of action transition probabilities; for instance, the first equation assigns the ratio $\frac{f_B^0}{f_A^0} = \frac{\frac{1}{8}}{\frac{1}{4}} = \frac{1}{2}$, which corresponds to the ratio of probabilities of actions B and A to be triggered without delay. Finally, equations in rule **(C)** assign probabilities to delay transitions of more than one action; as an example, the second equations assign probability $(1 - \phi_A^0) * (1 - \phi_B^2) = (1 - \frac{1}{4}) * (1 - \frac{1}{3}) = \frac{1}{2}$, which corresponds to the multiplication of the probabilities to delay A by 1 tick and to delay B by 3 ticks. The TPTS in Fig 5 is obtained by solving the equations (note that probabilities are rounded to 3 decimal places for brevity).

5 Implementation

The implementation of our approach, named `ptsv`, and the case study presented in the next section are publicly available in our repository[1]

5.1 The `ptsv` tool

The `ptsv` tool takes a network of DSTA as input and returns a TPTS. Optionally, users can specify a network of TA and a set of traces to obtain TPTS by injecting the traces into the TLTS

[1] `https://github.com/irmanfaqrizal/ptsv`

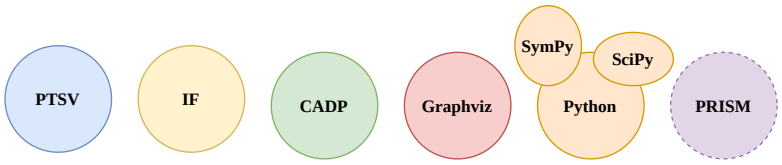


Fig. 6: `ptsv` and external tools

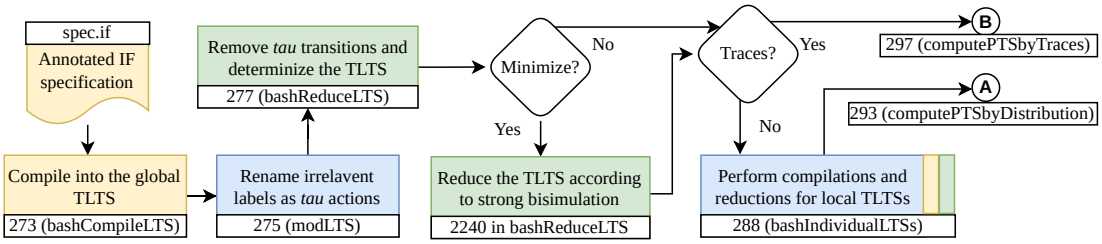


Fig. 7: Preprocessing of TLTS in `ptsv`

compiled from the network of TA. The tool is written in Java, and it utilises several external tools, including `IF` [2], `CADP` [9], `SciPy` [19], and `Graphviz` [7]. The usages of `IF` and `CADP` are explained in the previous sections. As for `SciPy`, it is a Python-based mathematics framework that we use to solve systems of equations. Optionally, the equations can also be solved using the symbolic mathematics framework `SymPy` [16]. `Graphviz` is used to produce a PDF file of the TPTS. The `ptsv` tool has been tested on a Linux system, and these external tools must be installed before running it. In Fig, 6, these tools are associated with specific colors to facilitate the explanation of their usage. The PRISM tool [13] is used to compute steady-state probabilities for verification purposes. The workflow of the tool is segmented into three parts: preprocessing, TPTS computation, and output construction.

Preprocessing. Fig, 7 illustrates the preprocessing part of the tool. Every task in the illustration is annotated with line numbers (referring to its location in the code) and function names. Also, files are annotated with their (example) names and extensions.

The preprocessing part takes as input a subset of the `IF` specification language [3], annotated with probabilistic delay distributions. More precisely, users may annotate *informal actions* in the specification with $[dist]$, where $dist ::=$ **uniform** | **custom:**F. The **uniform** annotation assigns a discrete uniform distribution to the transition delay associated with the action, whereas **custom** allows users to specify a sequence of fractional numbers F.

The `IF` and `CADP` tools are used to obtain a global TLTS by compiling the specification and applying reduction techniques. If a set of traces is given as input, then the tool starts the TPTS computation by injecting the traces. Without the traces, the tool first compiles and preprocesses the local TLTSs for generating TPTS by solving a system of equations.

TPTS computation. TPTS can be computed according to probabilistic delay distributions or by injection of traces. Fig, 8 shows the computation of TPTS according to probabilistic delay distributions. The first 12 tasks required to generate a system of equations as described in Section 4. Afterwards, depending on the input command, the user may choose to either solve the equations symbolically or numerically.

Fig, 9 shows the computation of TPTS by injecting a set of traces. A trace is a finite sequence of actions annotated with timing information. In practice, the `ptsv` tool takes a trace in the

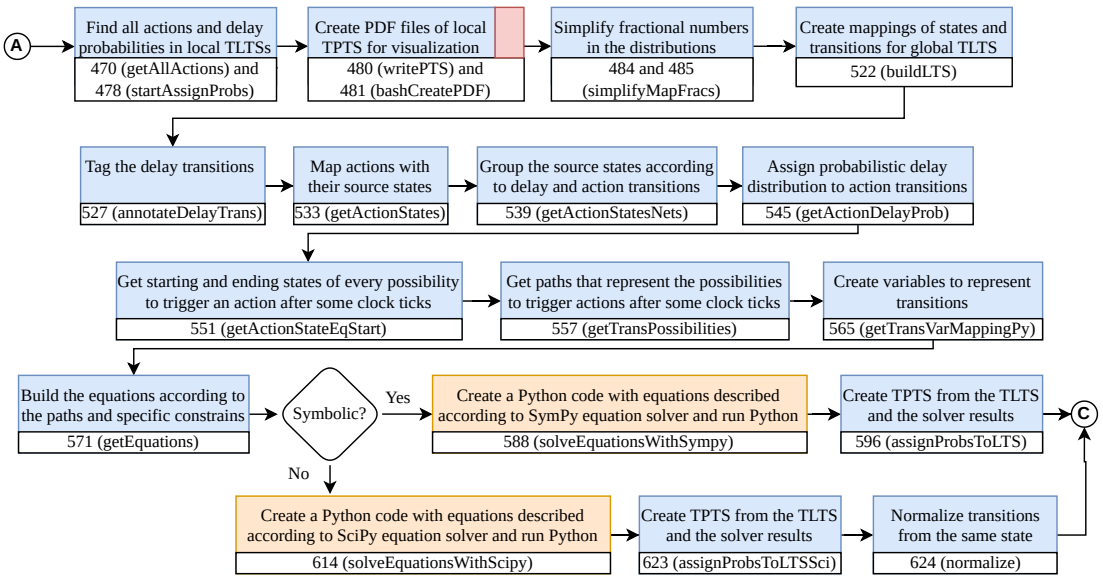


Fig. 8: TPTS computation according to probabilistic delay distribution in `ptsv`

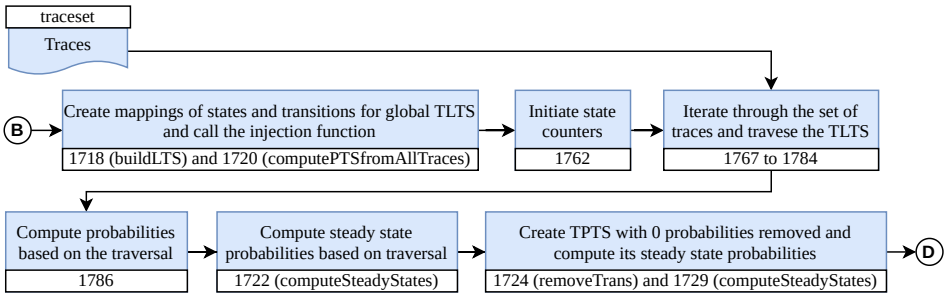


Fig. 9: TPTS computation by trace injections

following form:

$$a_1 \ b_1, a_2 \ b_2, ..., a_n \ b_n,$$

where $a_i \in A$ is an action, $b_i \in \mathbb{N}$ indicates the number of clock ticks passing since a_{i-1} is triggered (or since the start of the execution if $i = 1$), and $n \in \mathbb{N}$ is the length of the trace.

Outputs. The outputs of `ptsv` depend on the method for computing the TPTSs. Fig, 10 shows the outputs when the TPTS is computed according to probabilistic delay distributions. The format `.aut` is a textual representation of the TPTS. We use *Graphviz* to also generate the PDF file of the TPTS. When a symbolic solver is chosen, the tool also returns the TPTS with probabilities represented as fractional numbers. However, these files can only be used for visualization since `CADP` only recognizes decimal probabilities. The tool also returns a Discrete Time Markov Chain (DTMC) that can be used to compute steady state probabilities for verification purposes.

Fig, 11 shows the outputs when the TPTS is obtained by injecting traces. In this case, the tool returns two types of TPTS. The first type *spec-min-traceset-pts.aut* and *spec-min-traceset-pts.pdf* consists of all the original states and transitions according to the initial global TLTS. The second

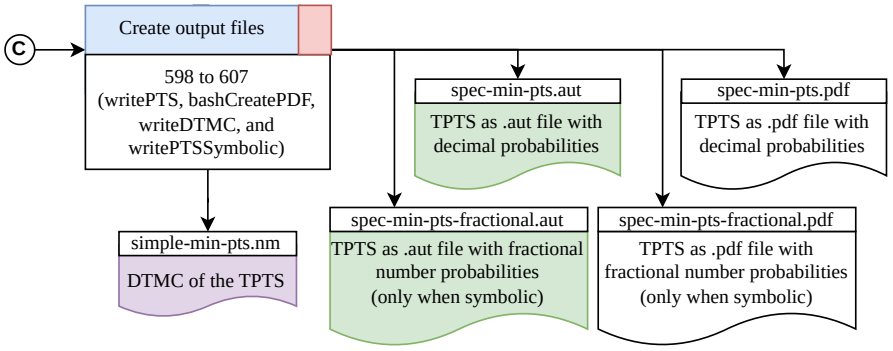


Fig. 10: Outputs of TPTS computation according to probabilistic delay distributions

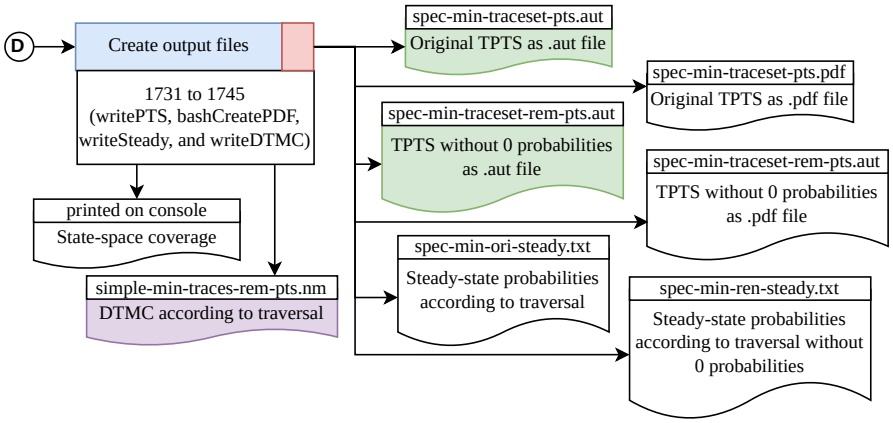


Fig. 11: Outputs of trace injections

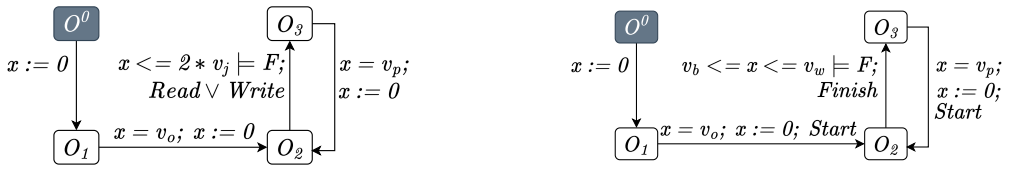
type *spec-min-traceset-rem-pts.aut* and *spec-min-traceset-rem-pts.pdf* consists of only reachable states and transitions, which means that transitions with 0 probabilities and their subsequent states and transitions are removed. This second type of TPTS is required for verification because `CADP` forbids transitions with 0 probabilities, and the first type can be used for visualization and state-space analysis purposes (see next section for example). In addition to the TPTSs, the tool also returns a DTMC according to the traversal. Moreover, the state-space coverage of the traversal is also printed on the console for performing coverage analysis.

5.2 Performance evaluation

We evaluated the performance of our tool by performing experiments on small examples, and the result is presented in Table 3. DSTA with locations and edges as presented in Fig. 2 are used, with the periodicity fixed to be 30 (on edge O_2 to O_1) and activation delay $x <= |F| - 1$ (on edge O_1 to O_2) for all the automata. The experiments are parameterized by the number of DSTA and $|F|$ (cols 2 and 3), which result in various numbers of states, transitions in the global TLTS, and equations to be solved (cols 4-6). We are interested in comparing the computation time, which is split into (local and global) TLTS generations, system of equations construction, and solving times (cols 7-9). While the construction of the system of equations scales linearly with the state space, the solving time currently represents the primary bottleneck.

Table 3: Performance evaluation

	Specification		State-space			Computation (ms)		
	DSTA	$\|F\|$	$\|S\|$	$\|T\|$	$\|Eqs\|$	T. TLTSs	T. Equations	T. Solver
1.	2	2	8	13	13	4167	766	325
2.	2	4	16	29	27	4155	758	303
3.	2	6	24	45	41	4282	752	332
4.	3	2	16	33	34	5626	986	328
5.	4	2	32	81	83	7083	1270	436
6.	3	4	32	73	72	5696	1028	429
7.	4	4	64	177	177	7248	1308	3614
8.	3	6	48	133	110	5685	1035	935
9.	4	6	96	273	271	7178	1577	10391



(a) Camera and brake

(b) AEB controller

Fig. 12: Network of DSTA

6 Case study: Automated Emergency Braking System

This section demonstrates the usability of our approach with the Automated Emergency Braking System (AEBS), a vehicle application designed to mitigate collisions. Generally, the system consists of (i) sensors, such as cameras, radars, and lidars, (ii) controllers for data fusion and time-to-collision computation, and (iii) actuators associated with the brake and the alarm. We consider a simplified version of AEBS where a camera captures data and sends it to an AEB controller, which in turn computes new data and forwards it to the brake actuator. These components activate periodically.

System Specification. The DSTA network of the system is depicted in Fig. 12. The camera and brake have the same behaviour, so they also have the same locations and edges, thus the same automaton (Fig. 12a). The only difference is that in the camera, the periodic action *Read* means reading from the environment, whereas action *Write* in the brake means writing data to the hardware component. The automaton describes that the component activates periodically according to v_p with jitter $2 * v_j$ and offset v_o. On the other hand, the automaton for the AEB controller (Fig. 12b) describes that the component starts executing (i.e., action *Start*) periodically according to v_p with offset v_o (but without jitter), and can finish (i.e., action *Finish*) between v_b and v_w. In both automata, $\models F$ means that the transition delay satisfies the discrete probabilistic distribution expressed as a sequence of fractional numbers F.

Scenario and Trace Injection. The objective of the case study is to show that the generated state-space of the specification can be used as a reference model when comparing simulation and execution results. To achieve this objective, we first generate the TPTS of the DSTA network in Fig. 12 with our method; this TPTS is called *golden TPTS*. Afterwards, we perform two simulations. The first one represents a simulation that the developer performs at design-time to analyse the system response time. The second simulation is considered to be the actual system

Table 4: Experimental timing configurations

Conf.	Camera				AEB controller					Brake			
	v_o	v_p	v_j	F	v_o	v_p	v_b	v_w	F	v_o	v_p	v_j	F
C1	4	10	1	$\left[\frac{7}{9}, \frac{1}{9}, \frac{1}{9}\right]$	10	5	1	3	$\left[\frac{1}{9}, \frac{7}{9}, \frac{1}{9}\right]$	4	10	1	$\left[\frac{7}{9}, \frac{1}{9}, \frac{1}{9}\right]$
C2	4	10	1.5	$\left[\frac{1}{7}, \frac{9}{14}, \frac{1}{14}\ \frac{1}{7}\right]$	10	5	1	3	$\left[\frac{1}{6}, \frac{2}{3}, \frac{1}{6}\right]$	4	10	0.5	$\left[\frac{8}{9}, \frac{1}{9}\right]$

Table 5: State-space coverage

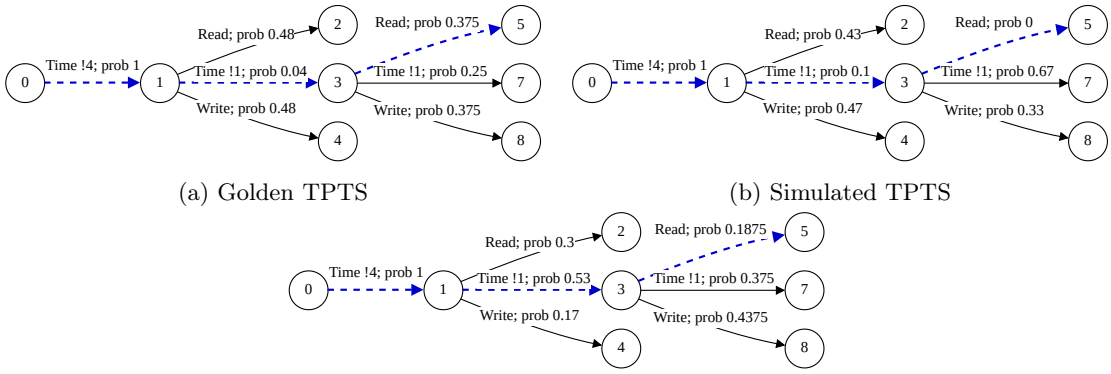
TPTS	C1				C2			
	States	Transitions	S. Cov	T. Cov	States	Transitions	S. Cov	T. Cov
Golden	46	76	100%	100%	44	69	100%	100%
Simulation	43	64	94%	84%	44	61	100%	88%
Execution	43	65	94%	86%	44	63	100%	91%

execution. Simulations produce traces, sequences of actions annotated with discrete timestamps. The traces are collected to be injected into the TLTS so that two additional TPTSs are obtained: the *simulated TPTS* (according to simulation traces) and *executed TPTS* (according to system execution traces). The trace injection method is described in [8]. Essentially, the TLTS is traversed according to the trace, and the probability of transition is obtained by dividing the number of times that transition is traversed by the number of times its source state is visited.

Experimental Configuration. We tested the scenario with two configurations shown in Table 4. The simulations are performed using the discrete-event simulator of the `Simpy`[2] framework. Each simulation and configuration is executed 30 times, and up to 100 clock ticks each. To produce different simulation and execution traces, we modify F to be a discrete uniform distribution when simulating the actual execution (i.e., to generate traces for computing the executed TPTS). Once the three TPTSs are generated, we demonstrate two types of analysis. The first one aims at investigating the representativeness of the simulation and execution. This is done by checking the state-space coverage. In addition, we investigate transitions with 0 probabilities in the resulting simulated and executed TPTSs. Paths leading to these transitions are extracted to obtain sequences of actions and clock ticks that were not observed during the simulations, and then we examine the golden TPTS to determine the expected probabilities of these sequences. The second analysis relies on the action-based probabilistic model checking [14] of the `CADP` toolbox [9]. More precisely, we define an MCL5 formula [15] that serves as input to the model checker to extract response-time probabilities from the generated TPTSs. The formula utilises the iteration operator to find paths consisting of actions *Read*, *Start*, *Finish*, and *Write*, sequentially, while counting the number of ticks in between.

State-space Analysis. The state-space coverage of simulated and executed TPTSs is presented in Table 5. In the table, we present the number of reachable states and the number of transitions with probabilities greater than 0. Also, S.Cov and T. Cov mean state and transition coverages, respectively. Our method for computing the state-space of timed automata with

[2]https://simpy.readthedocs.io/



(a) Golden TPTS

(b) Simulated TPTS

(c) Executed TPTS

Fig. 13: Fragments of TPTSs generated according to configuration C1

uncertainties successfully assigned probabilities to all transitions in the TPTSs. Therefore, the golden TPTSs for both configurations cover 100% of states and transitions. In contrast, both simulated and executed TPTSs have transitions with 0 probabilities, which leads to unreachable states in some cases. This means that there are sequences of actions and clock ticks that should be possible according to the specification, but did not happen during the simulations. Compared to simulated TPTS, executed TPTS has slightly more coverage because its transition delay probabilities have a discrete uniform distribution.

We investigate further transitions with 0 probabilities in simulated and executed TPTSs by extracting paths leading to them. This extraction is done using the `EXHIBITOR` tool from the `CADP` toolbox. As an example, Fig. 13b shows a fragment of the simulated TPTS from configuration C1 with a highlighted path leading to a transition with 0 probability. In this path, the sequence *Time* !*4*, *Time* !*1*, *Read* describes that after 4 ticks, which corresponds to the offset v_o in C1, both actions *Read* and *Write* are delayed by 1 tick before *Read* is triggered. In Fig 13a, the corresponding path in the golden TPTS is presented. It shows that, even though it is small (i.e., 0.015), the probability of that path is greater than 0 according to the specification. On the other hand, the same path in the executed TPTS seen in Fig. 13c has a larger probability due to the discrete uniform distribution of the transition delay.

Ultimately, the state-space analysis gives insight to the developers about the representativeness of the simulation with the state-space coverage. In this example, the developers are informed that the simulation did not cover all possible sequences in the model. In addition, the extraction of paths leading to 0 probability transitions further informs developers about edge cases that were not observed during simulation. In this case, for instance, the developers may conclude from the results that it is not surprising that a specific sequence was not observed in the simulation because the corresponding path in the golden TPTS is improbable. In contrast, the actual execution should be investigated because the probability of the path in executed TPTS (see Fig. 13c is suspiciously greater (i.e., close to 0.1) than the one according to the specification (i.e., 0.015). This discrepancy acts as a diagnostic trigger, suggesting that the hardware implementation may have a scheduling bias or a resource contention issue not captured by the original DSTA distribution, but visible through the TPTS comparison.

Response Time Analysis. By applying probabilistic queries on the generated TPTSs using the `CADP` toolbox, response time probabilities depicted in Fig. 14 are obtained. As seen in the result for configuration C1, there is a small probability of the sequence *Read*, *Start Finish*, and *Write* to happen after only 2 ticks. This case is not observed during simulation or execution,

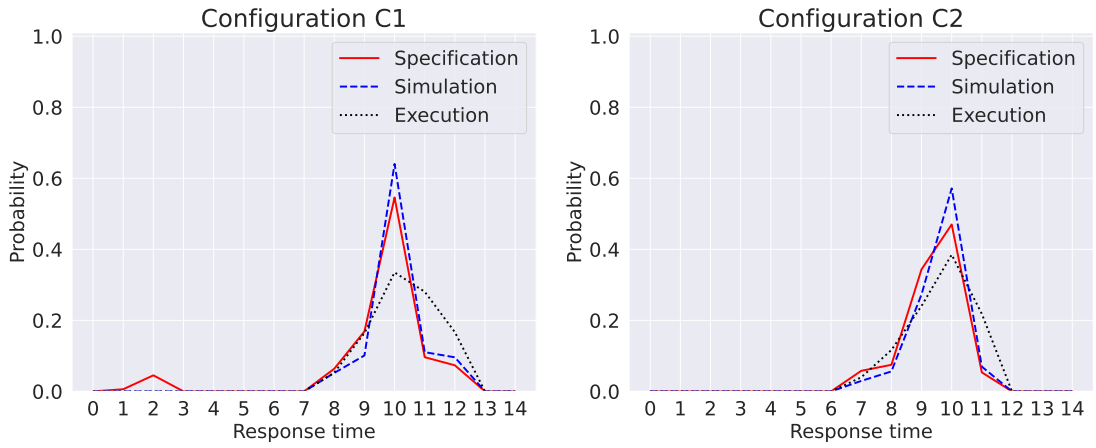


Fig. 14: Response time probabilities

which may be interpreted in different ways. One possible interpretation is that such a short response time could be considered not ideal; thus, the specification may need to be adjusted by modifying the component periodicities. Furthermore, if the developers consider that the system must execute as close as possible to the specification, having this analysis is useful to compare the results of simulation and execution. For instance, in configuration C2, the simulation response time probabilities are relatively similar compared to those from the execution result.

Note that many other properties can be quantitatively analysed due to the expressiveness of the MCL5 formula. For instance, it is possible to obtain the probabilities of activating the actuator without new data received by the controller (e.g., *Read* followed by another *Read* without *Finish* in between). Having the probabilistic models generated according to the specification, simulation, and actual execution enables various queries for comparing any specific temporal behaviours in the models that can be expressed as sequences of actions.

7 Conclusion

This paper proposed a formal methodology to bridge the gap between specification, simulation, and execution in real-time systems with timing uncertainties. By transforming a network of Discrete Stochastic Timed Automata (DSTA) into a Timed Probabilistic Transition System (TPTS), we provide a framework for consistent stochastic analysis. The generation process relies on a system of linear equations derived from the equivalence between local delay distributions and global interleavings. Our case study on an Automated Emergency Braking System (AEBS) demonstrates that the resulting "Golden TPTS" acts as a critical reference for validating the representativeness of simulation and execution traces. Furthermore, the integration with the `CADP` toolbox allows for the extraction of complex timing properties, such as response-time distributions, via MCL5 queries. Future work will focus on extending the DSTA model to include transition probabilities for modeling non-deterministic failures and supporting synchronized communication for event-triggered components.

Acknowledgments. This work is supported by the European Chips Joint Undertaking under Framework Partnership Agreement No 101139789 (HAL4SDV).

References

[1] Rajeev Alur and David L. Dill. A theory of timed automata. *Theor. Comput. Sci.*, 126(2): 183–235, 1994. doi:10.1016/0304-3975(94)90010-8.

[2] Marius Bozga, Susanne Graf, Ileana Ober, Iulian Ober, and Joseph Sifakis. The IF toolset. In *Proceedings of SFM-RT 2004*, volume 3185 of *LNCS*, pages 237–267. Springer, 2004. doi:10.1007/978-3-540-30080-9_8.

[3] Marius Bozga, Susanne Graf, and Laurent Mounier. *The IF Language Reference Manual*. VERIMAG Laboratory, 2021. URL `https://www-verimag.imag.fr/~async/IF/tutorials.html`. Accessed: 2026-02-01.

[4] Peter E. Bulychev, Alexandre David, Kim Guldstrand Larsen, Marius Mikucionis, Danny Bøgsted Poulsen, Axel Legay, and Zheng Wang. UPPAAL-SMC: statistical model checking for priced timed automata. In *Proceedings of QAPL 2012*, volume 85 of *EPTCS*, pages 1–16, 2012. doi:10.4204/EPTCS.85.1.

[5] Krishnendu Chatterjee and Thomas A. Henzinger. Value iteration. In *25 Years of Model Checking - History, Achievements, Perspectives*, volume 5000 of *LNCS*, pages 107–138. Springer, 2008. doi:10.1007/978-3-540-69850-0_7.

[6] Pedro R. D'Argenio, Holger Hermanns, Joost-Pieter Katoen, and Ric Klaren. Modest - A modelling and description language for stochastic timed systems. In *Proceedings of PAPM-PROBMIV 2001*, volume 2165 of *LNCS*, pages 87–104. Springer, 2001. doi:10.1007/3-540-44804-7_6.

[7] John Ellson, Emden Gansner, Lefteris Koutsofios, Stephen North, and Gordon Woodhull. Graphviz — open source graph drawing tools. In *Graph Drawing*, volume 2265 of *LNCS*, pages 483–484. Springer, 2002. doi:10.1007/3-540-45848-4_59.

[8] Irman Faqrizal, Gwen Salaün, and Yliès Falcone. Probabilistic analysis of industrial iot applications. In *Proceedings of IoT 2022*, pages 41–48. ACM, 2022. doi:10.1145/3567445.3567461.

[9] Hubert Garavel, Frédéric Lang, Radu Mateescu, and Wendelin Serwe. CADP 2011: a toolbox for the construction and analysis of distributed processes. *Int. J. Softw. Tools Technol. Transf.*, 15(2):89–107, 2013. doi:10.1007/S10009-012-0244-Z.

[10] Ernst Moritz Hahn, Arnd Hartmanns, and Holger Hermanns. Reachability and reward checking for stochastic timed automata. *Electron. Commun. Eur. Assoc. Softw. Sci. Technol.*, 70, 2014. doi:10.14279/TUJ.ECEASST.70.968.

[11] Arnd Hartmanns and Holger Hermanns. The modest toolset: An integrated environment for quantitative modelling and verification. In *Proceedings of TACAS 2014*, volume 8413 of *LNCS*, pages 593–598. Springer, 2014. doi:10.1007/978-3-642-54862-8_51.

[12] Arnd Hartmanns and Holger Hermanns. Explicit model checking of very large MDP using partitioning and secondary storage. In *Proceedings of ATVA 2015*, volume 9364 of *LNCS*, pages 131–147. Springer, 2015. doi:10.1007/978-3-319-24953-7_10.

[13] Marta Z. Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *Proceedings of CAV 2011*, volume 6806 of *LNCS*, pages 585–591. Springer, 2011. doi:10.1007/978-3-642-22110-1_47.

[14] Radu Mateescu and José Ignacio Requeno. On-the-fly model checking for extended action-based probabilistic operators. *Int. J. Softw. Tools Technol. Transf.*, 20(5):563–587, 2018. doi:10.1007/S10009-018-0499-0.

[15] Radu Mateescu and Damien Thivolle. *MCL: A Model Checking Language for Concurrent Systems*. INRIA, 2021. URL `https://cadp.inria.fr/man/mcl5.html`. Part of the CADP toolbox documentation.

[16] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Tim Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Anthony. Sympy: symbolic computing in python. *PeerJ Computer Science*, 3:e103, January 2017. ISSN 2376-5992. doi:10.7717/peerj-cs.103. URL `https://doi.org/10.7717/peerj-cs.103`.

[17] Xavier Nicollin and Joseph Sifakis. An overview and synthesis on timed process algebras. In *Proceedings of CAV 1991*, volume 575 of *LNCS*, pages 376–398. Springer, 1991. doi:10.1007/3-540-55179-4_36.

[18] Michael Paulweber, Andreas Eckel, and Paolo Azzoni. Multi-partner project: Shaping the vehicle of the future - how FEDERATE and HAL4SDV are steering europe's software-defined vehicle ecosystem. In *Proceedings of DATE 2025*, pages 1–4. IEEE, 2025. doi:10.23919/DATE64628.2025.10992971.

[19] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi:10.1038/s41592-019-0686-2.

[20] Håkan L. S. Younes and Reid G. Simmons. Probabilistic verification of discrete event systems using acceptance sampling. In *Proceedings of CAV 2002*, volume 2404 of *LNCS*, pages 223–235. Springer, 2002. doi:10.1007/3-540-45657-0_17.

RESEARCH CENTRE
Centre Inria d'Université Côte d'Azur

2004 route des Lucioles - BP 93
06902 Sophia Antipolis Cedex

Publisher
Inria
Domaine de Voluceau - Rocquencourt
BP 105 - 78153 Le Chesnay Cedex
inria.fr

ISSN 0249-0803